MW REALTY · TESTER HANDBOOK · PROJECT DOCUMENT P8

Test Automation Tools

What automated testing exists in the project today, and the tools we recommend for the QA team's automation.

Scope: Automated tests in the code repository, and the tools for browser (E2E), API, performance, accessibility and device testing of the website, Admin and CRM. Tool choices are recommendations; the QA team may use equivalents it already licenses.

1. What exists today

Item	Status in the repository
Back-end tests	PHPUnit 12 (`phpunit/phpunit`), Laravel's test runner (`php artisan test`)
Test files	28 feature test files in `tests/Feature` and 2 unit test files in `tests/Unit`
Areas covered	Admin security, ads, agency / agent management and plans, contact form, Facebook leads, KYC notifications, lead assignment and round robin, listing compliance (permits), viewings, dashboard, reports, two-factor, watermark, property contact, Property Finder, list filters and map, property options, sold listings, translations, shared master data, sitemap, support tickets, URL redirect safety
Test database	`phpunit.xml` sets SQLite in memory (`DB_CONNECTION=sqlite` , `DB_DATABASE=:memory:`), mail to `array` , queue `sync` , cache `array`
Front-end / browser tests	None (no Playwright, Cypress, Dusk, Vitest or Jest)
`data-testid` attributes in Vue screens	None
CI pipeline	No CI workflow in the repository
API definitions	Scribe exports a Postman collection and an OpenAPI file per API: `/docs/_scribe/frontend/` , `/docs/_scribe/crm/` , `/docs/_scribe/app-customer/` , `/docs/_scribe/app-agent/` (`collection.json` , `openapi.yaml`)

Running the PHP tests: always use `composer test` . It runs `php artisan config:clear` first. If the configuration is cached, `php artisan test` can ignore `phpunit.xml` and run against the real development database, and the tests then wipe it. Never run the tests on the QA, UAT or production server.

2. Recommended tool stack

Tool	Use	Why it fits this app
Playwright (TypeScript)	Browser E2E for website, Admin and CRM	Website and CRM are Vue single-page apps; Playwright waits for network and UI by itself, runs Chromium, Firefox and WebKit (Safari engine), emulates phones, and can test Arabic right-to-left layouts

Tool	Use	Why it fits this app
Postman + Newman (or Bruno)	API functional and contract tests	Import the Scribe `collection.json` / `openapi.yaml` and start with every endpoint already defined; Newman runs the collections in CI
k6 (or JMeter)	Load tests on search, property details, compare and lead forms	Script-based, easy to run in CI; respect the API rate limits listed in P7 or tests will see HTTP 429
axe-core (via Playwright) and Lighthouse	Accessibility and performance checks	Runs inside the E2E suite; Lighthouse for page speed of home, search and property details
BrowserStack or LambdaTest	Real-device checks	Safari on iPhone and Chrome on Android, at the widths listed in P6
Stripe CLI	Payment webhooks in QA	`stripe listen --forward-to` delivers test-mode events to the QA server (see P7)
Mail catcher (Mailpit or a shared SMTP test inbox)	Sign-up codes, 2FA and notification emails	Lets automated tests read one-time codes without server log access (see P3)

3. Suggested automation scope

Order	Suite	Contents	Source of cases
1	Smoke (every deployment)	Website home, search, property details, enquiry form; CRM login with 2FA, dashboard, leads list, add listing; Admin login	High priority scenarios in guides 00, 02, 07
2	Role and access	Each role opens allowed pages and is refused (403) on others	Section 3 of each module guide, P3 permission matrix
3	Regression by module	High, then Medium scenarios per module guide, tagged with the scenario ID (for example `@LP-08`)	Section 5 of guides 01–26
4	API	Status codes, required fields, validation messages, auth and rate limits	P7 endpoint list, Scribe collections
5	Cross-browser and mobile	Smoke suite on WebKit, Firefox and mobile emulation; Arabic RTL	P6 browser and device matrix
6	Performance	Search and property details under load	P7 rate limits, agreed targets (to be confirmed with the client)

Keep Low priority (cosmetic) scenarios manual.

4. Practical guidance

- **Selectors:** the Vue screens have no `data-testid` attributes. Use Playwright role and label locators (`getByRole`, `getByLabel`, visible text). Ask the development team to add `data-testid` to key controls (login, listing form, lead actions, approve / reject buttons) to make tests stable; Arabic text changes text-based selectors.
- **Test data:** create data through the UI or API at the start of each test, with names from P4 (for example "Test Agency QA 01"). Do not depend on records other testers change.

- **Codes and email:** two-factor and sign-up codes arrive by email. Use the shared test inbox or the fixed QA code described in P3 (ask the project lead).
- **Background work:** imports, photo screening and notifications run on the queue. Tests must wait for the result (poll the status screen or API) rather than assume it is instant; the queue worker must run on QA (P6).
- **Time and schedules:** daily jobs (permit expiry, reminders) run on a schedule; test them by asking a developer to run the command (P7) instead of waiting.
- **Reporting:** link each automated test to its scenario ID so failures map straight to the module guide; attach the Playwright trace and screenshot to bug reports.
- **CI:** run the smoke and API suites on every deployment to QA, the full regression nightly. Store credentials as CI secrets, never in the test code.